



曦云[®] 系列通用 GPU

MXMACA[®] Clangd 用户手册

OG-24072-020-F5_V02

2025-03-21

声明

版权所有 ©2024-2025 沐曦集成电路（上海）股份有限公司。保留所有权利。

本文档中呈现的信息属于沐曦集成电路（上海）股份有限公司和/或其附属公司（以下统称为“沐曦”），非经沐曦事先书面许可，任何实体或个人均不得获得本文档的副本，且无权以任何方式处理本文档，包括但不限于使用、复制、修改、合并、出版、发行、销售或传播本文档的部分或全部。

本文档内容仅供参考，不提供任何形式的、明示或暗示的保证，包括但不限于对适销性、适用于任何目的和/或不侵权的保证。在任何情况下，沐曦均不对因本文档引起的、由本文档造成的、或与之相关的任何索赔、损害或其他责任负责。

沐曦保留自行决定随时更改、修改、添加或删除本文档的部分或全部的权利。沐曦保留最终解释权。

沐曦、MetaX 和其他沐曦图标是沐曦的商标。本文档中提及的所有其他商标和商品名称均为其各自所有者的财产。

目录

1 概述	1
1.1 工作流程	1
1.2 关键特性	1
1.3 支持平台	2
1.3.1 Clangd 工具	2
1.3.2 Clangd 插件	2
2 MXMACA Clangd 安装	3
2.1 准备文件	3
2.2 安装 Clangd 插件	3
2.3 禁用 VSCode 官方 C/C++ 插件（若有）	3
3 MXMACA Clangd 使用	5
3.1 为工程生成 compile_commands.json 文件	5
3.2 独立文件或简单工程	6
4 特性列表	7
4.1 .maca 文件识别	7
4.2 错误和告警诊断	7
4.3 代码补全	8
4.4 跳转	9
4.5 工程导航	10
4.6 悬停	11
4.7 代码规范化	11
4.8 代码重构	12
4.9 关键字高亮	15
4.10 源文件与头文件切换	15
4.11 显示 AST	15
4.12 显示 MXMACA 预处理后文件	17
5 进阶配置	18
5.1 配置 Clangd 工具路径	18
5.2 重启 Clangd 工具	18

图目录

图 1.1	MXMACA Clangd 工作流程	1
图 2.1	安装 Clangd 插件	3
图 2.2	点击配置按钮	4
图 2.3	选择禁用插件	4
图 2.4	禁用生效	4
图 3.1	.maca 文件示例	5
图 4.1	.maca 文件识别	7
图 4.2	实时显示编译报错和告警信息	7
图 4.3	代码自动修复	8
图 4.4	提供全称候选列表	8
图 4.5	插入头文件	8
图 4.6	头文件插入成功	9
图 4.7	函数参数提示	9
图 4.8	跳转	9
图 4.9	OUTLINE 栏目显示	10
图 4.10	搜索本文件或本工程	10
图 4.11	显示函数调用链	10
图 4.12	悬停显示	11
图 4.13	选择代码规范化	11
图 4.14	代码修复成功	11
图 4.15	选择重命名符号	12
图 4.16	输入新变量名称	12
图 4.17	重命名成功	13
图 4.18	抽取	13
图 4.19	选择抽取表达式为变量	14
图 4.20	抽取为变量成功	14
图 4.21	选择抽取语句为函数	14
图 4.22	抽取为函数成功	14
图 4.23	展开宏	14
图 4.24	展开宏成功	15
图 4.25	源文件与头文件切换	15
图 4.26	选择显示 AST	16
图 4.27	显示抽象语法树	16
图 4.28	显示 MXMACA 预处理后文件	17
图 4.29	预处理后文件窗口	17
图 5.1	配置 Clangd 工具路径	18

更新记录

版本	日期	更新说明
V02	2025-03-21	更新以下章节： 4.12 显示 MXMACA 预处理后文件
V01	2024-12-20	首次发布

1 概述

MXMACA® Clangd 是支持 MXMACA C/C++ 语言的编辑器辅助工具集，包括支持语言服务协议（LSP）的 Clangd 工具（以下称为 Clangd 工具）和编辑器插件（以下称为 Clangd 插件）。目前支持 VSCode 编辑器，提供了 VSCode 插件。

在编辑器中编写、阅读 MXMACA C/C++ 代码时，MXMACA Clangd 可以提供文件识别、代码补全、跳转、诊断等多种功能，具有功能全面、识别准确、响应速度快等优点，帮助使用者快速、高效地开发、浏览 MXMACA C/C++ 工程。

1.1 工作流程

如下图所示，在安装了 VSCode 插件和 MXMACA 的环境中，用户通过 VSCode 执行某个功能，Clangd 插件发送对应的消息到 Clangd 工具，Clangd 工具处理后，将处理结果再通过消息返回给 Clangd 插件，经由 VSCode 显示给用户。

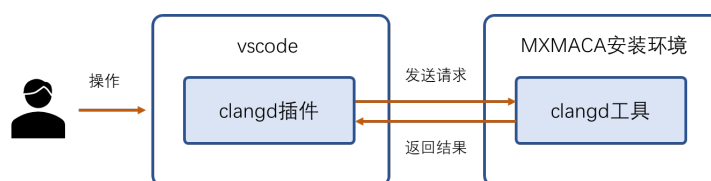


图 1.1 MXMACA Clangd 工作流程

1.2 关键特性

MXMACA Clangd 支持以下特性：

- 支持 MXMACA C/C++ 工程，支持自动识别 **.maca** 后缀的文件为 MXMACA C/C++ 语言，并设置对应图标
- 兼容 C/C++ 语言，支持 MXMACA C/C++ 和 C/C++ 混合工程
- 支持 MXMACA C/C++ 语言关键字、MXMACA 头文件和 API 的识别、跳转
- 支持语言服务协议规定的大部分特性，支持了一些协议以外的扩展特性
- 与官方 C/C++ Extension 插件相比，整体上功能更多，响应速度更快

1.3 支持平台

1.3.1 Clangd 工具

Clangd 工具支持的 CPU 架构和操作系统同 MXMACA SDK，具体参见《曦云® 系列通用 GPU 用户指南》中“系统支持范围”章节。

注解：Clangd 工具支持情况不涉及编辑器。

1.3.2 Clangd 插件

编辑器：目前支持 Visual Studio Code。

注解：Clangd 插件支持情况不涉及架构和 OS，使用的是支持的编辑器即可。

2 MXMACA Clangd 安装

2.1 准备文件

下载并安装 2.27.0 及以后版本的 MXMACA SDK 包。

Clangd 工具已安装于 `/opt/maca/mxgpu_llvm/bin/clangd`。Clangd 插件安装文件位于 `/opt/maca/mxgpu_llvm/extensions/packages/MXMACA-clangd-0.2.0.vsix`。

2.2 安装 Clangd 插件

采用 VSCode 插件离线安装方法，将 Clangd 插件安装到 VSCode。

1. 在 VSCode 主界面选择 **View -> Extensions**。
2. 点击 **Views and More Actions** 图标 ，选择 **Install from VSIX**，找到 **MXMACA-clangd-0.2.0.vsix** 文件进行安装。部分环境中安装耗时可能较长，耐心等待即可。

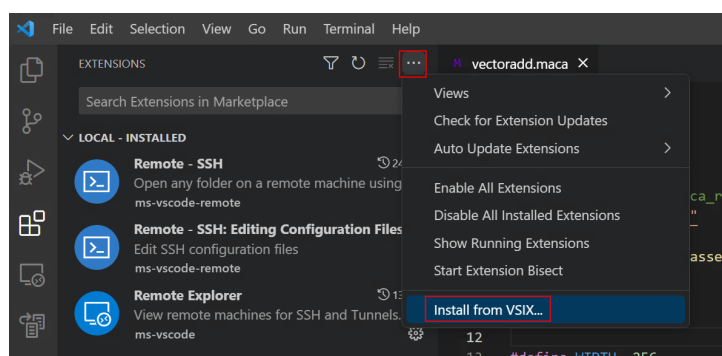


图 2.1 安装 Clangd 插件

2.3 禁用 VSCode 官方 C/C++ 插件（若有）

Clangd 和官方 C/C++ 插件会有冲突，若已安装 C/C++ 插件，如 C/C++ Extension Pack，需禁用。

1. 点击插件右侧配置按钮 。

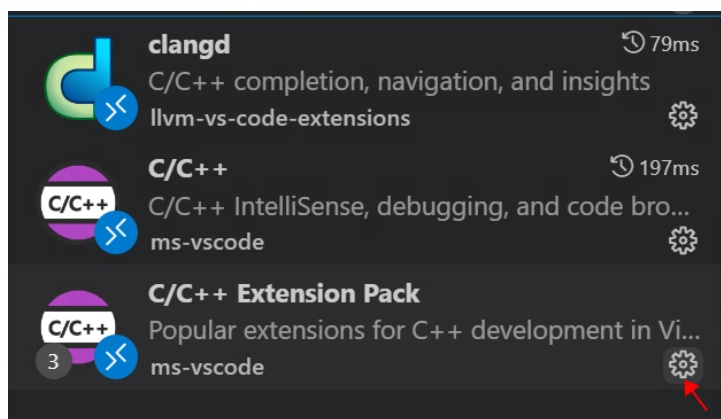


图 2.2 点击配置按钮

2. 根据需要选择 **Disable** 或 **Disable (Workspace)**，在全局或当前工程禁用插件。

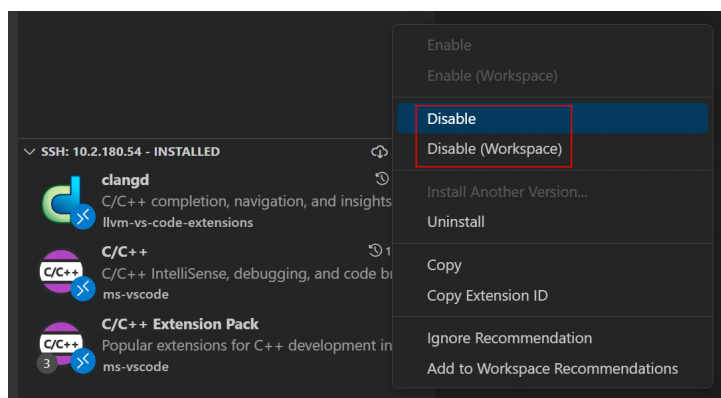


图 2.3 选择禁用插件

3. 点击插件的 **Reload Required** 或直接重启 VSCode 使得禁用生效，即插件变灰色。

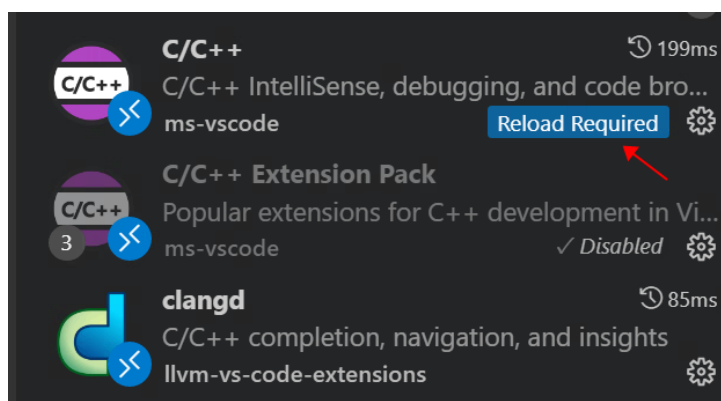


图 2.4 禁用生效

3 MXMACA Clangd 使用

打开工程目录或文件即可使用。正常情况下，能够识别 **.maca** 文件，代码高亮显示，无编译报错波浪线提示，如图 3.1 所示。具体功能详见 4 特性列表。

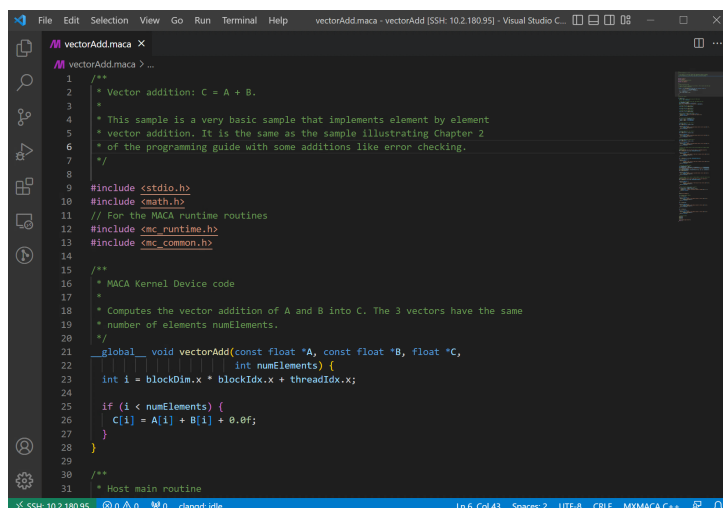


图 3.1 .maca 文件示例

体验完整功能需要提供准确的编译命令，比如准确的变量跳转、头文件包含、预编译宏分支选择等。用户需要提供一个含有每个文件编译命令的 **compile_commands.json** 的文件，供 Clangd 获取编译命令。该文件通常可以自动生成，详见 3.1 为工程生成 **compile_commands.json** 文件。

MXMACA 相关头文件需要从 MXMACA 安装路径中获取，Clangd 依次从编译命令的 **-maca-path** 选项、环境变量 **MACA_PATH**、Clangd 所在目录、**/opt/maca** 中获取 MXMACA 安装路径。

3.1 为工程生成 **compile_commands.json** 文件

基于 **cmake** 的工程

- 方式 1：在 **cmake** 命令中添加 **-DCMAKE_EXPORT_COMPILE_COMMANDS=1**
- 方式 2：在 **CMakeLists.txt** 文件中添加 **set (CMAKE_EXPORT_COMPILE_COMMANDS ON)**

在构建根目录下将生成 **compile_commands.json**，Clangd 会自动搜索到该文件。

基于 **make** 的工程

使用 **bear** 工具，根据版本不同，命令为 **bear make** 或 **bear -- make**。

其他构建工程，或使用其他工具生成该文件，可参考 [工程构建相关文档](#)。

Clangd 工具自动搜索文件目录和其下的 **build** 目录，以及递归搜索父目录和其下的 **build** 目录。如果上述目录中没有这个文件，可以软链接或手动拷贝过去。

3.2 独立文件或简单工程

Clangd 支持没有 **compile_commands.json** 文件的情况，此时 Clangd 默认其编译命令为 `mxcc <file>`。对单个文件或包含关系比较简单的工程，此时也能有相对完整的功能体验。

如果需要在默认命令上添加一些编译选项，可以使用 **compile_flags.txt** 配置文件，参考 [compile_flags.txt 相关文档](#)。

4 特性列表

4.1 .maca 文件识别

识别 .maca 后缀为 MXMACA 语言，图标为 ，VSCode 状态栏显示为 **MXMACA C++**，如图 4.1 所示。

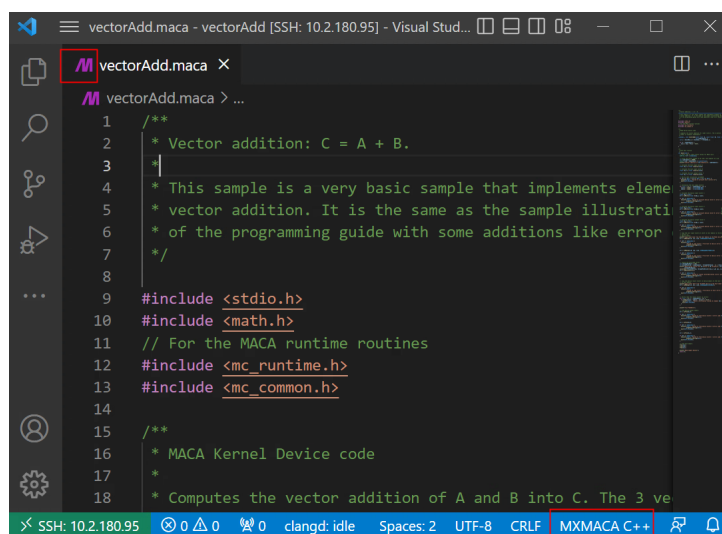


图 4.1 .maca 文件识别

4.2 错误和告警诊断

- 编写代码时，实时显示编译报错和告警信息。

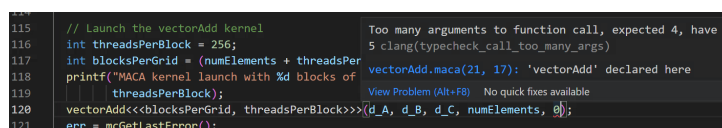


图 4.2 实时显示编译报错和告警信息

- 自动修复

提示正确代码，点击后可自动修复。如图 4.3 所示，点击  图标后，可以选择修正错误的函数名。

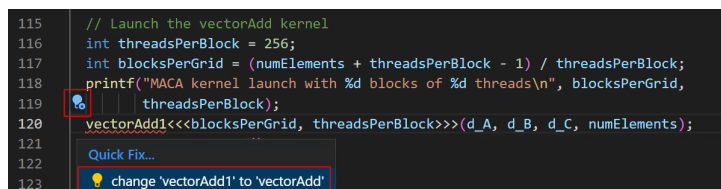


图 4.3 代码自动修复

- clang-tidy 检查

集成了 clang-tidy，可在编程规范、错误等方面提供更充分的检查。具体使用方式参考 [Clang-Tidy 相关文档](#)。

4.3 代码补全

- 编写代码时，给出可能的全称候选列表，供快速输入

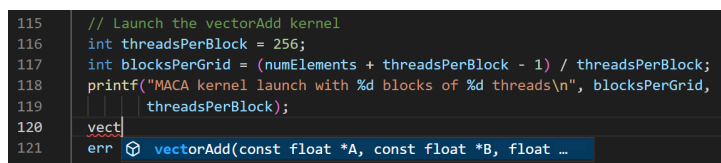


图 4.4 提供全称候选列表

- 命名空间和头文件自动插入

编写代码时，可以自动插入命名空间或头文件。对于插入命名空间，键入变量或函数时，在候选列表中可以看到带有完整命名空间的变量或函数。对于插入头文件，键入足够信息后回车确认，即可将头文件插入顶部。

例如如图 4.5 所示，键入 `std::string` 后，回车确认，可以按字母顺序自动插入其所属的 C++ 标准库头文件 `<string>` 到文件顶部。头文件插入成功后如图 4.6 所示。

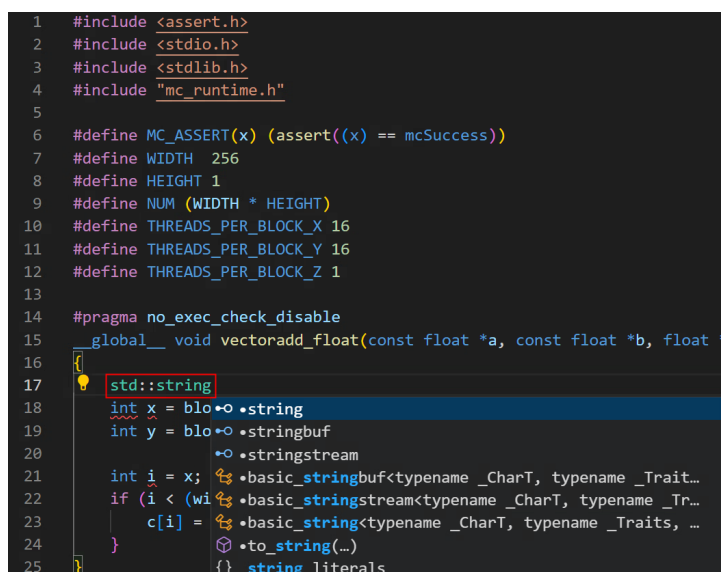


图 4.5 插入头文件

```
1  #include <assert.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string>
5  #include "mc_runtime.h"
6
7  #define MC_ASSERT(x) (assert((x) == mcSuccess))
8  #define WIDTH 256
9  #define HEIGHT 1
10 #define NUM (WIDTH * HEIGHT)
11 #define THREADS_PER_BLOCK_X 16
12 #define THREADS_PER_BLOCK_Y 16
13 #define THREADS_PER_BLOCK_Z 1
14
15 #pragma no_exec_check_disable
16 __global__ void vectoradd_float(const float *a, const float *b, f
17 {
18     std::string
19     int x = blockDim.x * blockIdx.x + threadIdx.x;
20     int y = blockDim.y * blockIdx.y + threadIdx.y;
```

图 4.6 头文件插入成功

- 函数参数提示

编写代码时，给出调用函数的入参信息。

```
93 // Copy the host input vectors A and B in host memory to the device input
94 // vectors in
95 // device memory
96 printf("Copy input data from the host memory to the MACA device\n");
97 err = mcMemcpy
98     mcMemcpy(void *dst, const void *src, size_t coun...
99 if (err != mcS mcErrorInvalidMemcpyDirection
100     fprintf(stde mcMemcpyFromSymbolAsync(void *dst, const void *s...
101     "Fai mcMemcpyFromSymbol(...) !\n",
102     mcGe mcGraphMemcpyNodeSetParamsFromSymbol(mcGraphNode...
103     exit(EXIT_FA mcGraphAddMemcpyNodeFromSymbol(mcGraphNode_t *pG...
104 }
```

图 4.7 函数参数提示

4.4 跳转

查找到定义、声明、引用等，包括 MXMACA 安装包中的头文件，以及 MXMACA 的各类 API、宏、类型的声明或定义。如图 4.8 所示，在感兴趣的变量、函数上单击右键打开菜单，可选择对应的条目。

```
62
63 // Allocate the device input vector A
64 float *d_A = NULL;
65 err = mcMalloc((void **) &d_A, size);
66
67 if (err != mcS
68     fprintf(stde
69     mcGe
70     exit(EXIT_FA
71 }
72
73
```

图 4.8 跳转

4.5 工程导航

- 文件大纲

OUTLINE 栏目显示文件大纲，便于浏览并跳转到本文件的所有符号。如图 4.9 所示，在 **EXPLORER** 窗口打开 **Outline**，即可看到该栏目。

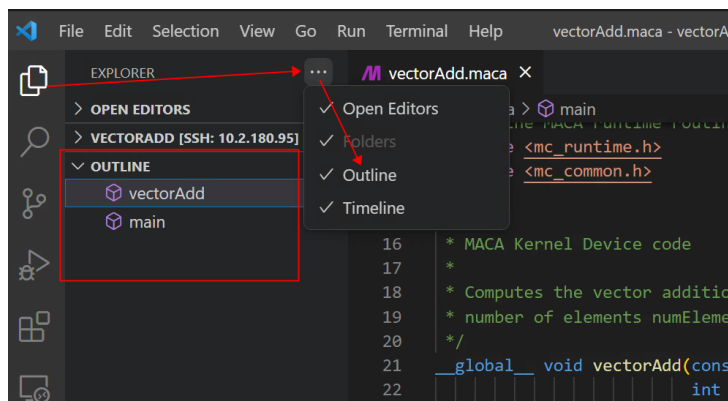


图 4.9 OUTLINE 栏目显示

- 搜索符号

Ctrl+P 打开搜索栏，如图 4.10 所示，输入 @ 搜索本文件，或输入 # 搜索本工程的符号。

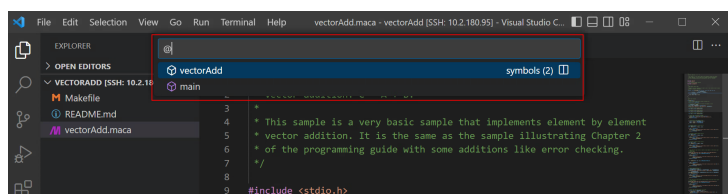


图 4.10 搜索本文件或本工程

- 显示当前函数的调用关系

如图 4.11 所示，在函数上右键打开菜单，选择 **Show Call Hierarchy**，左侧窗口中会出现调用链。

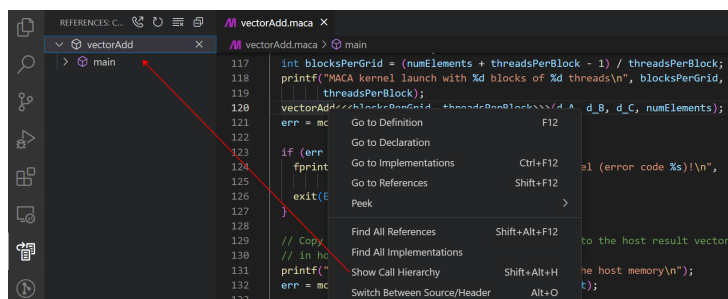


图 4.11 显示函数调用链

4.6 悬停

鼠标悬停时显示符号更多的信息，如图 4.12 所示。

```

51 // Verifv that allocations succeeded
52 if (h_
53     fpri function mcMalloc
54     exit
55 }
56     - mcError_t
57     Parameters:
58 // Ini
59 for (i
60     * void ** ptr
61     h_A[]
62     * size_t bytesize
63     h_B[]
64     @brief Allocate memory on the device
65     @param[out] ptr Pointer to allocated device memory
66     @param[in] bytesize Requested allocation size in bytes
67     @return #mcSuccess, #mcErrorMemoryAllocation (bad context, not enough memory),
68     float
69     #mcErrorInvalidValue (null *ptr)
70     err = mcMalloc((void **) &d_A, size);
71

```

图 4.12 悬停显示

4.7 代码规范化

集成了 clang-format，可以对代码格式进行规范化处理。如图 4.13 所示，选中一部分代码后，在右键菜单选择 **Format Selection**，可以将第 25 行自动修复为缩进 2 格，修复后如图 4.14 所示。也可以选择 **Format Document**，对整个文件进行处理。

clang-format 的使用详情可参考 [ClangFormat 相关文档](#)。

```

15 /**
16  * MACA Kernel Device code
17  *
18  * Computes the vector addition
19  * number of elements numElements
20  */
21 global void vectorAdd(const float *A, const float *B, float *C,
22                       int numElements) {
23     int i = blockDim.x * blockIdx.x + threadIdx.x;
24     if (i < numElements) {
25         C[i] = A[i] + B[i] + 0.0f;
26     }
27 }
28
29 /**
30  * Host main routine
31  */
32 int main(void) {
33     // Error code to check return
34     mcError_t err = mcSuccess;
35     // Print the vector length to
36

```

右键菜单选项：

- Go to Definition F12
- Go to Declaration
- Go to Implementations Ctrl+F12
- Go to References Shift+F12
- Peek >
- Find All References Shift+Alt+F12
- Find All Implementations
- Show Call Hierarchy Shift+Alt+H
- Switch Between Source/Header Alt+O
- Rename Symbol F2
- Change All Occurrences Ctrl+F2
- Format Document Shift+Alt+F
- Format Document With...
- Format Selection Ctrl+K Ctrl+F
- Refactor... Ctrl+Shift+R

图 4.13 选择代码规范化

```

20
21 global void vectorAdd(const float *A, const float *B, float *C,
22                       int numElements) {
23     int i = blockDim.x * blockIdx.x + threadIdx.x;
24
25     if (i < numElements) {
26         C[i] = A[i] + B[i] + 0.0f;
27     }
28 }
29

```

图 4.14 代码修复成功

4.8 代码重构

• 重命名符号

在任一需修改处选择 **Rename Symbol**，输入新符号名，所有引用的地方都会自动修改。

1. 在任一 `h_A` 变量处打开右键菜单，点击 **Rename Symbol**。

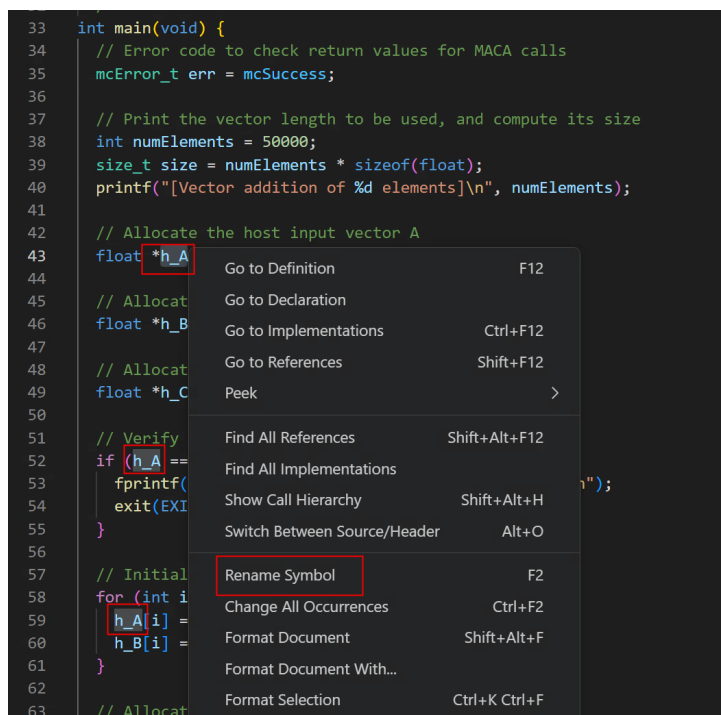


图 4.15 选择重命名符号

2. 在文本框中输入新名称 `hostA`。

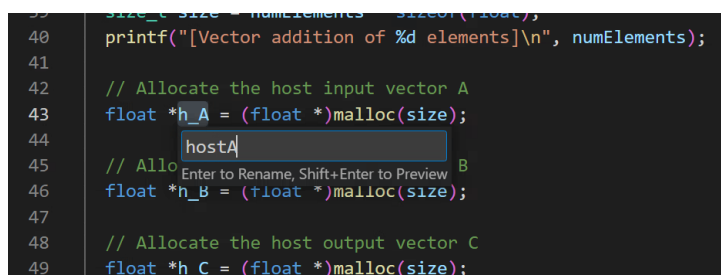


图 4.16 输入新变量名称

3. 回车确认后，所有的 `h_A` 都修改为 `hostA`。

```

33 int main(void) {
34     // Error code to check return values for MACA calls
35     mcError_t err = mcSuccess;
36
37     // Print the vector length to be used, and compute its size
38     int numElements = 50000;
39     size_t size = numElements * sizeof(float);
40     printf("[Vector addition of %d elements]\n", numElements);
41
42     // Allocate the host input vector A
43     float *hostA = (float *)malloc(size);
44
45     // Allocate the host input vector B
46     float *h_B = (float *)malloc(size);
47
48     // Allocate the host output vector C
49     float *h_C = (float *)malloc(size);
50
51     // Verify that allocations succeeded
52     if (hostA == NULL || h_B == NULL || h_C == NULL) {
53         fprintf(stderr, "Failed to allocate host vectors!\n");
54         exit(EXIT_FAILURE);
55     }
56
57     // Initialize the host input vectors
58     for (int i = 0; i < numElements; ++i) {
59         hostA[i] = rand() / (float)RAND_MAX;
60         h_B[i] = rand() / (float)RAND_MAX;
61     }
62

```

图 4.17 重命名成功

- 更多重构方式

支持抽取表达式为变量、抽取多行语句为函数、展开宏或 auto 类型。如图 4.18 所示，选中对应内容后，在右键菜单选择 **Refactor...**，根据提示进行后续操作。

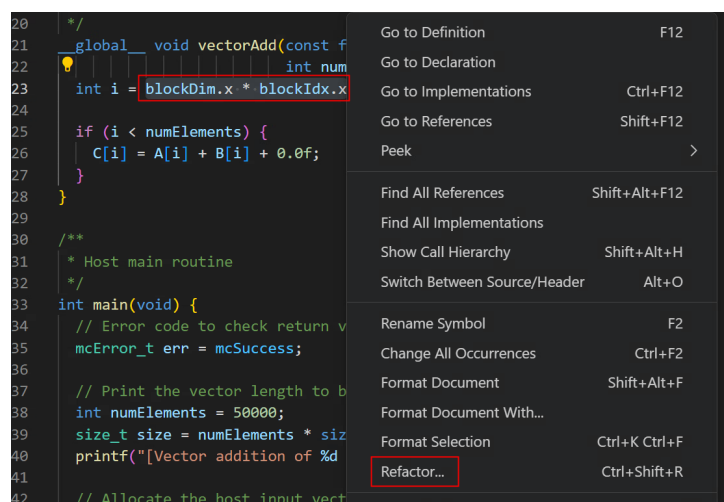


图 4.18 抽取

- 抽取表达式为变量：

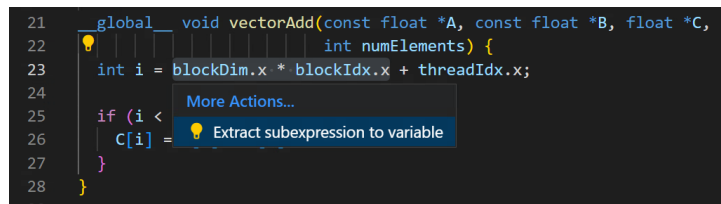


图 4.19 选择抽取表达式为变量

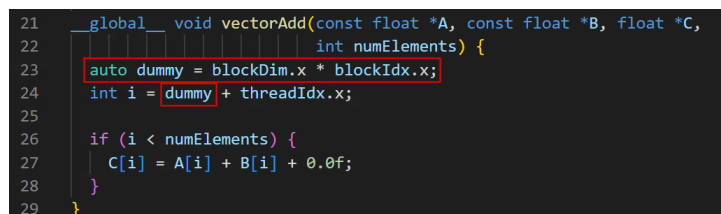


图 4.20 抽取为变量成功

– 抽取语句为函数：

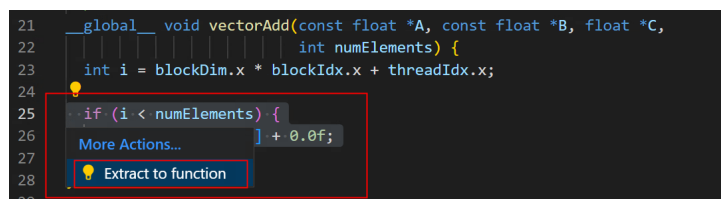


图 4.21 选择抽取语句为函数

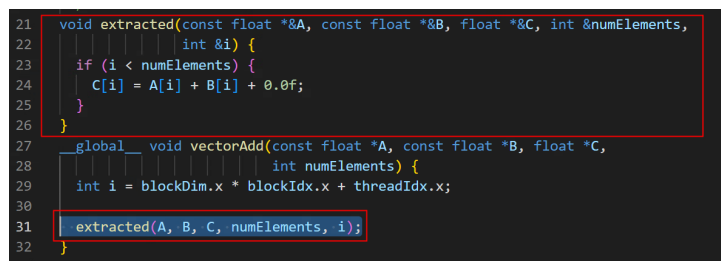


图 4.22 抽取为函数成功

– 展开宏：

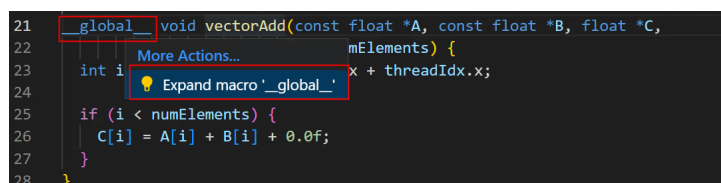


图 4.23 展开宏

```
21 | __attribute__((global)) void vectorAdd(const float *A, const float *B, float *C,  
22 |                                     int numElements) {  
23 |     int i = blockDim.x * blockIdx.x + threadIdx.x;  
24 |  
25 |     if (i < numElements) {  
26 |         C[i] = A[i] + B[i] + 0.0f;  
27 |     }  
28 | }
```

图 4.24 展开宏成功

4.9 关键字高亮

除了常规的 C/C++ 语言外，也支持 MXMACA C/C++ 语言关键字、宏等高亮。

4.10 源文件与头文件切换

支持在源文件和头文件之间快速切换。如图 4.25 所示，在源文件任意位置点击右键，在菜单中选择 **Switch Between Source/Header**：

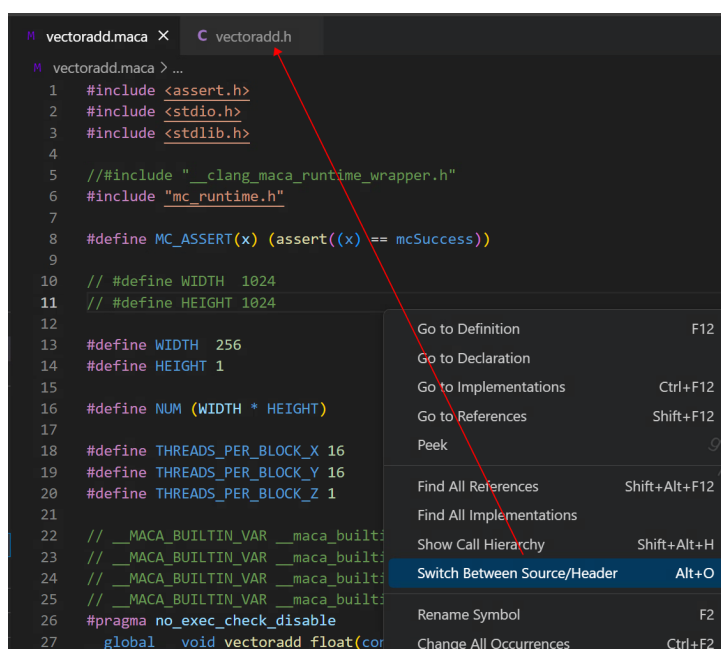


图 4.25 源文件与头文件切换

4.11 显示 AST

支持显示某段代码的抽象语法树信息，辅助开发者编写、理解代码。如图 4.26 所示，选中一段代码后，在右键菜单选择 **Show AST**，左侧显示抽象语法树如图 4.27 所示。

4.12 显示 MXMACA 预处理后文件

支持显示 device 侧预处理后文件。如图 4.28 所示，在右键菜单选择 **Show MXMACA Preprocessor Output**，在新的编辑窗口生成预处理后文件，同时右下角会弹出编译命令，如图 4.29 所示，供开发者核对。

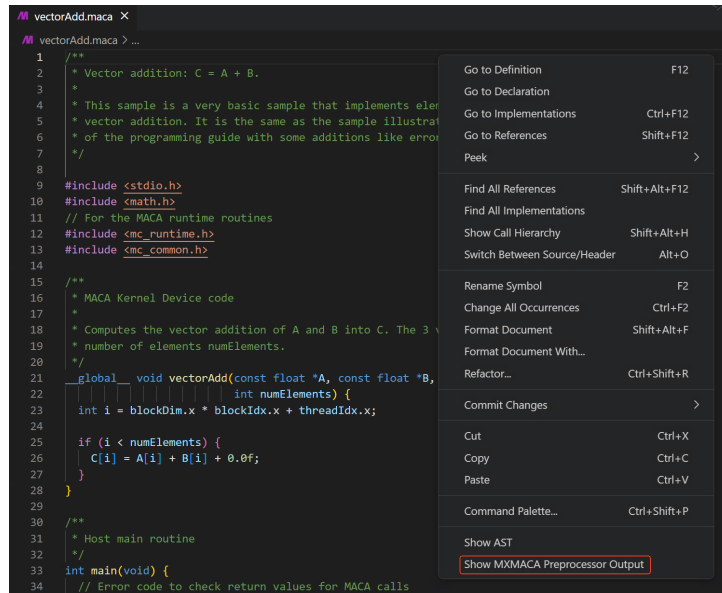


图 4.28 显示 MXMACA 预处理后文件

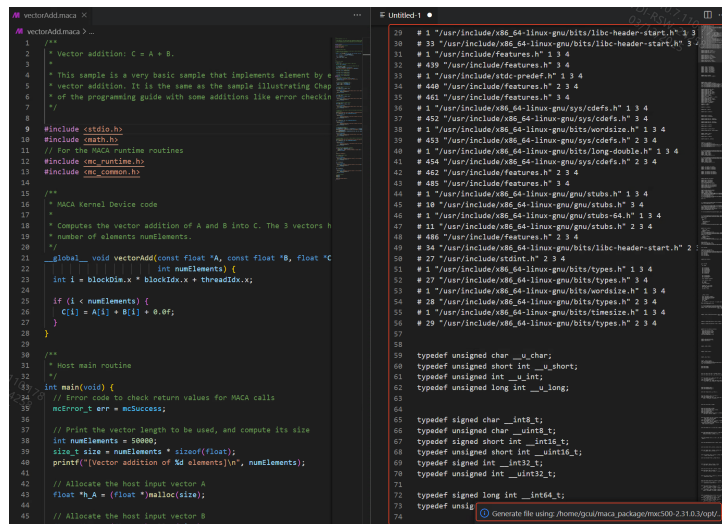


图 4.29 预处理后文件窗口

5 进阶配置

5.1 配置 Clangd 工具路径

如果需要用其他 MXMACA 目录下的 Clangd，可以在 Clangd 插件中设置对应路径。如图 5.1 所示，点击 Clangd 插件配置按钮 ，并选择 **Extension Settings**，根据需要，选择 **User**、**Remote** 或 **Workspace** 页面，配置 **Clangd: Path** 为所需的 Clangd 路径。

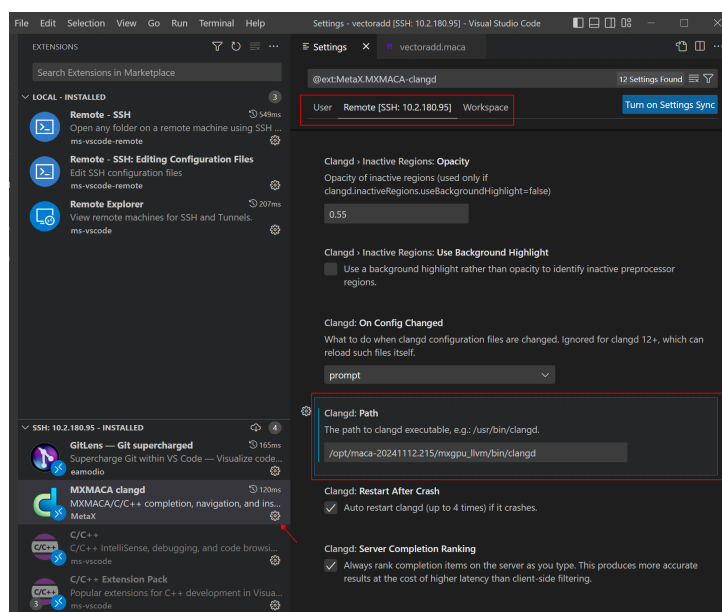


图 5.1 配置 Clangd 工具路径

5.2 重启 Clangd 工具

修改了 Clangd 插件的设置后，需要重启 Clangd 工具才能生效。按 **Ctrl + Shift + P**，输入 `clangd: Restart language server`，并执行，稍等即可生效。